

Can you give me small example for GANs working by hand

I'll walk you through a **tiny, by-hand example** of how a GAN works — using made-up numbers and a simple 1D data distribution.

1. Setup: What we're trying to do

We have:

- **Real data:** Just two possible points: $x = 2$ and $x = 3$.
So the real data distribution is basically 50% at 2 and 50% at 3.
- **Generator (G):** Starts with random noise z (say from a normal distribution with mean 0, variance 1), and tries to produce fake data $G(z)$ that looks like real data.
- **Discriminator (D):** Takes an input x (real or fake) and outputs a probability (0 to 1) that x is real.

Goal: Generator wants to fool discriminator; discriminator wants to tell real from fake.

2. Architecture (Simplified)

Let's fix a tiny **Generator function** for this example:

$$G(z) = w_g \cdot z + b_g$$

Initially, let $w_g = 1.0$, $b_g = 0.0$, so initially $G(z) = z$ (not good yet).

Discriminator function:

$$D(x) = \sigma(w_d \cdot x + b_d)$$

where $\sigma(t) = \frac{1}{1+e^{-t}}$ (sigmoid).

Initially, let $w_d = 1.0$, $b_d = 0.0$.

3. Step-by-step by hand

Step 1: Sample real data

We sample one real data point: $x_{\text{real}} = 2$.

Step 2: Generate fake data

Sample noise $z = 0.5$ (from $N(0, 1)$ but example value).

Fake data:

$$x_{\text{fake}} = G(z) = 1.0 \cdot 0.5 + 0.0 = 0.5$$

Step 3: Discriminator on real data

$$D(2) = \sigma(1.0 \cdot 2 + 0.0) = \sigma(2.0)$$

Sigmoid(2.0) ≈ 0.881 .

So D thinks probability real = 0.881 (should be high $\rightarrow$ good initially).

Step 4: Discriminator on fake data

$$D(0.5) = \sigma(1.0 \cdot 0.5 + 0.0) = \sigma(0.5) \approx 0.622$$

So D gives fake data a 0.622 probability of being real (not good — it's not great at detecting fake yet).

5. Discriminator update

Discriminator wants to **maximize** $D(x_{\text{real}})$ and **minimize** $D(x_{\text{fake}})$.

One common loss (binary cross-entropy):

$$L_D = -\log(D(x_{\text{real}})) - \log(1 - D(x_{\text{fake}}))$$

$$L_D \approx -\log(0.881) - \log(1 - 0.622)$$

$\approx -(-0.126) - \log(0.378)$ (wait, careful: $\log(0.881)$ is negative? Let's comp

Actually:

$$\log(0.881) \approx -0.126, \quad \log(0.378) \approx -0.973$$

So:

$$L_D = -[-0.126] - [-0.973] \quad ??$$

Hold on — formula is:

$$L_D = - \underbrace{[1 \cdot \log(D(x_{\text{real}})) + 0 \cdot \log(1 - D(x_{\text{real}}))]}_{\text{for real}} - \underbrace{[0 \cdot \log(D(x_{\text{fake}})) + 1 \cdot \log(1 - D(x_{\text{fake}}))]}_{\text{for fake}}$$


Wait, that's for two separate labels.

Actually simpler: For real (label=1): loss = $-\log(D(x_{\text{real}}))$.

For fake (label=0): loss = $-\log(1 - D(x_{\text{fake}}))$.

Total loss for D = sum.

So:

$$-\log(0.881) \approx -(-0.126) = 0.126$$

$$-\log(1 - 0.622) = -\log(0.378) \approx -(-0.973) = 0.973$$

Sum $L_D \approx 1.099$.

We'd compute gradients $\frac{\partial L_D}{\partial w_d}$ and $\frac{\partial L_D}{\partial b_d}$ and update w_d, b_d .

Let's skip derivative math, but imagine after update, maybe new $w_d = 1.2, b_d = -0.1$.

6. Generator update

Generator wants to **maximize** $D(G(z))$ (fool D), i.e., minimize $-\log(D(G(z)))$.

Using **original D before its update** (or sometimes same minibatch—depends), but for simplicity:

Generator loss:

$$L_G = -\log(D(x_{\text{fake}})) = -\log(0.622) \approx -(-0.474) = 0.474$$

Gradient of L_G w.r.t w_g, b_g :

We need $\frac{\partial L_G}{\partial D} \cdot \frac{\partial D}{\partial x_{\text{fake}}} \cdot \frac{\partial x_{\text{fake}}}{\partial w_g}$.

$$\frac{\partial L_G}{\partial D} = -\frac{1}{D(x_{\text{fake}})} = -\frac{1}{0.622} \approx -1.608$$

$$\frac{\partial D}{\partial x_{\text{fake}}} = D(1 - D) \text{ at } x_{\text{fake}} = 0.622 \times 0.378 \approx 0.235$$

Multiply: $-1.608 \times 0.235 \approx -0.378$.

Then $\frac{\partial x_{\text{fake}}}{\partial w_g} = z = 0.5$.

So $\frac{\partial L_G}{\partial w_g} \approx -0.378 \times 0.5 = -0.189$.

Update w_g with gradient ascent on $-L_G$ (or descent on L_G) — careful: actually generator wants to **maximize** $D(G(z))$, so loss = $-D(G(z))$ or $-\log D(G(z))$.

We used $-\log D$ loss, gradient w.r.t w_g we computed is ≈ -0.189 , so gradient ascent means:

$w_g \leftarrow w_g + \eta \cdot 0.189$ (since gradient of $-\log D$ w.r.t w_g = positive 0.189? Wait, we computed $\partial L_G / \partial w_g = -0.189$, but $L_G = +0.474$, so to minimize L_G we go opposite sign of gradient: $w_g \leftarrow w_g - \eta \cdot (-0.189) = w_g + \eta \cdot 0.189$).

Let's say $\eta=0.1$, then w_g becomes $1.0 + 0.0189 = 1.0189$.

Similarly update b_g .

7. Next round

New generator $G(z) = 1.0189z + (\text{new } b_g)$, new discriminator improved at detecting fakes if they remain around 0.5, but generator starts making outputs slightly larger.

Over many steps, generator's outputs should move toward real data distribution (around 2 or 3).

Key intuition

- **Discriminator** gets better at telling apart current fakes from reals.
- **Generator** adjusts to produce data that discriminator thinks is real.
- In equilibrium, generator produces exactly the real distribution: $x = 2$ or 3 with 50% each, and discriminator is confused (outputs 0.5 for any input).